



中山大学计算机学院

人工智能

本科生实验报告

(2024 学年春季学期)

课程名称: Artificial Intelligence

教学班级	人工智能 (饶洋辉)	专业 (方向)	23 级地理信息科学 (辅修)
学号	23306103	姓名	赵义凯

一、实验题目

使用 PyTorch 实现 DQN 算法训练 CartPole 智能体

二、实验内容

1. 算法原理

1.1 DQN 算法概述

DQN (Deep Q-Network) 结合深度神经网络与 Q-learning, 通过神经网络近似 Q 值函数, 解决传统 Q-learning 在高维状态空间下的存储和计算难题。其核心改进包括:

- 经验回放 (Replay Buffer):** 存储智能体与环境交互的经验元组 (s, a, r, s') , 缓解数据相关性和非平稳分布问题。
- 目标网络 (Target Network):** 引入独立的目标网络计算目标 Q 值, 降低训练过程中梯度更新的方差, 提升稳定性。

1.2 贝尔曼最优方程

Q-learning 的核心是贝尔曼最优方程, 描述如下:

$$Q^*(s, a) = \sum_{s' \in S} P_{s \rightarrow s'}^a \cdot (R_{s \rightarrow s'}^a + \gamma \max_{a' \in A} Q^*(s', a'))$$

其中, $Q^*(s, a)$ 表示状态 s 下执行动作 a 的最优 Q 值, r 为即时奖励, γ 为折扣因子, 用于平衡即时奖励与未来奖励。DQN 的优化目标是最小化预测 Q 值与目标 Q 值的均方误差, 目标 Q 值通过目标网络计算

1.3 Q-learning 理解

Q-learning 是一种离线策略 (Off-Policy) 的强化学习算法, 通过维护 Q 值表指导动作选择, 采用 ϵ -贪心策略平衡探索与利用。智能体在环境中执行动作, 收集经验并更新 Q 值表, 最终收敛至最优策略。DQN 通过神经网络替代 Q 值表, 实现对连续状态空间的泛化。

2. 伪代码



```
初始化行为网络 $Q(s; \theta)$ 和目标网络 $Q(s; \theta_-)$ 
初始化回放缓存池D, 容量为N
for episode in 1 to M:
    初始化状态s, 重置环境
    for step in 1 to T:
        根据 $\epsilon$ -贪心策略从 $Q(s; \theta)$ 选择动作a
        执行动作a, 获取奖励r、下一状态s'和终止标志done
        将(s, a, r, s', done)存入回放缓存池D
        从D中随机采样批量经验(batch_size)
        计算目标Q值:
            if done: y = r
            else: y = r +  $\gamma * \max(Q\_target(s'; \theta_-))$ 
        计算损失: L = MSE( $Q(s; \theta)[a]$ , y)
        反向传播更新行为网络参数 $\theta$ 
        每隔C步更新目标网络参数 $\theta_- = \theta$ 
        s = s'
    if done: break
    衰减 $\epsilon$ 值 ( $\epsilon = \max(\epsilon\_min, \epsilon * \epsilon\_decay)$ )
```

损失函数选择

采用均方误差 (MSE) 损失函数, 原因如下:

- 目标 Q 值与预测 Q 值均为连续值, MSE 适用于回归问题。
- Q-learning 的更新本质是监督学习, MSE 可衡量两者的数值差异, 与贝尔曼方程的迭代更新逻辑一致。

3. 关键代码展示 (带注释)

3.1 Q 网络定义 (QNetwork 类)

```
class QNetwork(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(QNetwork, self).__init__()
        # 输入层到隐藏层
        self.fc1 = nn.Linear(input_size, hidden_size)
        # 隐藏层到输出层 (动作空间维度)
        self.fc2 = nn.Linear(hidden_size, output_size)

    def forward(self, inputs):
        # 隐藏层使用ReLU激活函数
        x = F.relu(self.fc1(inputs))
        # 输出层直接返回Q值, 不使用激活函数
        return self.fc2(x)
```

3.2 回放缓存池 (ReplayBuffer 类)



```
class ReplayBuffer:
    def __init__(self, buffer_size):
        self.buffer = deque(maxlen=buffer_size) # 使用deque实现固定容量的缓存池

    def push(self, *transition):
        # 存储经验元组: (状态, 动作, 奖励, 下一状态, 终止标志)
        self.buffer.append(transition)

    def sample(self, batch_size):
        # 随机采样批量经验, 解压缩为独立列表
        samples = random.sample(self.buffer, batch_size)
        return zip(*samples)
```

3.3 智能体训练逻辑 (AgentDQN 类)

```
class AgentDQN(Agent):
    def train(self):
        episodes = self.args.episodes
        for ep in range(episodes):
            state, _ = self.env.reset()
            total_reward = 0
            done = False
            truncated = False
            while not done and not truncated:
                action = self.make_action(state) #  $\epsilon$ -贪心策略选择动作
                next_state, reward, done, truncated, _ = self.env.step(action)
                # 将经验存入回放缓存池
                self.buffer.push(state, action, reward, next_state, done)
                state = next_state
                total_reward += reward
                # 当缓存池数据足够时启动更新
                if len(self.buffer) >= self.batch_size:
                    self.update()
                self.steps += 1
                # 定期同步目标网络参数
                if self.steps % self.target_update_freq == 0:
                    self.target_net.load_state_dict(self.behavior_net.state_dict())
            # 衰减探索率 $\epsilon$ 
            if self.epsilon > self.epsilon_min:
                self.epsilon *= self.epsilon_decay
            # 保存训练好的行为网络参数
            self.behavior_net.save(self.save_path)
```

3.4 网络更新逻辑 (update 方法)

```
def update(self):
    # 从缓存池采样批量经验
    states, actions, rewards, next_states, dones = self.buffer.sample(self.batch_size)
    # 转换为PyTorch张量并移动至指定设备
    states = torch.FloatTensor(np.array(states)).to(self.device)
    actions = torch.LongTensor(actions).unsqueeze(1).to(self.device)
    # 计算当前Q值: 行为网络预测值
    current_q = self.behavior_net(states).gather(1, actions)
    # 计算目标Q值: 目标网络计算, 使用最大Q值
    with torch.no_grad():
        next_q = self.target_net(next_states).max(1, keepdim=True)[0]
        expected_q = rewards + (1 - dones) * self.gamma * next_q
    # 均方误差损失函数
    loss = F.mse_loss(current_q, expected_q)
    # 反向传播更新行为网络参数
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()
```

4. 创新点&优化（如果有）

三、 实验结果及分析

1. 实验参数设置

超参数名称	含义	设置值
env_name	实验使用的环境名称，这里选用了经典的 OpenAI Gym 环境。	CartPole-v1
train_dqn	布尔型标志，用于指示是否进行 DQN 模型的训练。	True
test	布尔型标志，用于指示是否进行模型测试。	False
seed	随机数种子，确保实验的可重复性。	11037
hidden_size	神经网络隐藏层的神经元数量，影响模型的表示能力。	32
lr	学习率，控制模型参数更新的步长。	0.001
gamma	折扣因子，平衡当前奖励和未来奖励的重要性。	0.99

超参数名称	含义	设置值
epsilon	探索率，决定智能体采取随机动作的概率。	0.2
tau	软更新参数，用于目标网络的更新。	1.0
use_cuda	布尔型标志，指示是否使用 CUDA 进行 GPU 加速。	True

2. gamma 值对训练的影响

- $\gamma=0.99$: 注重长期奖励，智能体倾向于学习维持杆平衡的长远策略。训练初期奖励增长较慢，但后期稳定性高，最终奖励可达 300+(CartPole-v1)。

```
Episode 532, Reward: 292.0, Epsilon: 0.014
Episode 533, Reward: 212.0, Epsilon: 0.014
Episode 534, Reward: 385.0, Epsilon: 0.014
Episode 535, Reward: 255.0, Epsilon: 0.014
Episode 536, Reward: 379.0, Epsilon: 0.014
Episode 537, Reward: 231.0, Epsilon: 0.014
Episode 538, Reward: 195.0, Epsilon: 0.014
Episode 539, Reward: 500.0, Epsilon: 0.013
Episode 540, Reward: 326.0, Epsilon: 0.013
```

- $\gamma=0.9$: 更关注即时奖励，训练初期奖励提升较快，但可能陷入局部最优，例如仅通过短期动作维持平衡，最终奖励难以达到稳定高分。

```
Episode 518, Reward: 224.0, Epsilon: 0.015
Episode 519, Reward: 226.0, Epsilon: 0.015
Episode 520, Reward: 221.0, Epsilon: 0.015
Episode 521, Reward: 237.0, Epsilon: 0.015
Episode 522, Reward: 227.0, Epsilon: 0.015
Episode 523, Reward: 231.0, Epsilon: 0.015
Episode 524, Reward: 217.0, Epsilon: 0.015
Episode 525, Reward: 245.0, Epsilon: 0.014
Episode 526, Reward: 311.0, Epsilon: 0.014
Episode 527, Reward: 238.0, Epsilon: 0.014
Episode 528, Reward: 269.0, Epsilon: 0.014
Episode 529, Reward: 224.0, Epsilon: 0.014
Episode 530, Reward: 240.0, Epsilon: 0.014
Episode 531, Reward: 259.0, Epsilon: 0.014
```

3. 目标网络的作用分析

- 使用目标网络: 训练过程稳定，损失函数波动较小，奖励曲线呈单调上升趋势，且能在一定轮次使得奖励值收敛到自己想要的区间。
- 不使用目标网络: 行为网络同时用于预测和目标计算，导致梯度更新震荡，损失函数剧烈波动，奖励增长缓慢且不稳定，甚至可能无法收敛至目标分



数。

```
Episode 219, Reward: 187.0, Epsilon: 0.067
Episode 220, Reward: 292.0, Epsilon: 0.067
Episode 221, Reward: 144.0, Epsilon: 0.066
Episode 222, Reward: 98.0, Epsilon: 0.066
Episode 223, Reward: 108.0, Epsilon: 0.066
Episode 224, Reward: 359.0, Epsilon: 0.065
Episode 225, Reward: 211.0, Epsilon: 0.065
Episode 226, Reward: 249.0, Epsilon: 0.065
Episode 227, Reward: 10.0, Epsilon: 0.064
Episode 228, Reward: 11.0, Epsilon: 0.064
Episode 229, Reward: 10.0, Epsilon: 0.064
Episode 230, Reward: 125.0, Epsilon: 0.063
Episode 231, Reward: 131.0, Epsilon: 0.063
Episode 232, Reward: 170.0, Epsilon: 0.063
Episode 233, Reward: 477.0, Epsilon: 0.063
Episode 234, Reward: 26.0, Epsilon: 0.062
Episode 235, Reward: 14.0, Epsilon: 0.062
```

4. 训练结果展示

最开始 500 轮次是第一次训练，测试奖励值为 191

```
PS D:\大二下\人工智能资料\实验课\Lab9\lab_9_DQN> python -u "d:\大
是否训练: False
是否测试: True
进入测试
Loading model from ./dqn_model.pth ...
Test total reward: 191.0
PS D:\大二下\人工智能资料\实验课\Lab9\lab_9_DQN>
```

发现效果不佳，进行第二次训练后再进行测试，得到下面结果，测试奖励值为 213，略有增高

```
> python -u "d:\大二下\人工智能资料\实验课\Lab9\lab_9_DQN
是否训练: False
是否测试: True
进入测试
Loading model from ./dqn_model.pth ...
Test total reward: 213.0
```

最后将轮次增加，发现训练过程中奖励值在 550 左右收敛了，于是我选择 550 作为最终的轮次，经过多次测试后得到 500 满分的奖励。

```
> python -u "d:\大二下\人工智能资料\实验课\Lab9\lab_9_DQN\main.py"
是否训练: False
是否测试: True
进入测试
Loading model from ./dqn_model.pth ...
Test total reward: 500.0
PS D:\大二下\人工智能资料\实验课\Lab9\lab_9_DQN>
```



四、 思考题

1. DQN 中目标网络的作用是什么？若不使用目标网络，训练过程会出现什么问题？

目标网络的作用是通过独立计算目标 Q 值，降低训练中梯度更新的方差，提升稳定性。若不使用目标网络，行为网络需同时用于预测和目标计算，会导致梯度更新震荡、损失函数剧烈波动，奖励增长缓慢且不稳定，甚至无法收敛至目标分数

2. 在 **Replay Buffer** 中，为什么需要随机采样批量经验？直接按顺序采样会有什么问题？

随机采样目的：缓解数据相关性和非平稳分布问题，使训练数据更符合独立同分布假设，提升模型泛化能力。

顺序采样问题：若按顺序采样，相邻经验可能高度相关（如连续动作的状态转移），导致梯度更新偏向局部模式，增加训练震荡风险，降低收敛效率。

五、 参考资料

[1] Lab9 参考文档

[2] [强化学习核心算法：深度 Q 网络（DQN）算法_dqn 算法-CSDN 博客](#)

[3] [强化学习 7—— 一文读懂 Deep Q-Learning（DQN）算法_deep q learning-CSDN 博客](#)